

[← Go Back](#)

Hardware

Portenta Vision Shield

Tutorials

[Blob Detection with Portenta and OpenMV](#)[Saving Bitmap Camera Images to the SD Card](#)[Connect Portenta Vision Shield to TTN with LoRa® Technology](#)[Creating a Basic Face Filter With OpenMV](#)[Training a Custom Machine Learning Model for Portenta H7](#)[Using the Portenta Vision Shield - Ethernet with Arduino IDE](#)[Using the Portenta Vision Shield - Ethernet with OpenMV](#)[Getting Started With the Portenta Vision Shield Camera](#)[Voice Commands With The Arduino Speech Recognition Engine](#)[Connecting to The Things Network Using OpenMV](#)[Portenta Vision Shield User Manual](#)

Home / Hardware / Portenta Vision Shield / [Saving Bitmap Camera Images to the SD Card](#)

Saving Bitmap Camera Images to the SD Card

This tutorial shows you how to capture a frame from the Portenta Vision Shield Camera module and save the output as a bitmap image.

Author · Pablo Marquínez · Last revision · 09/25/2024

ON THIS PAGE

Overview

[Goals](#) +[Instructions](#) +[Conclusion](#)[Next Steps](#)

Overview

This tutorial shows you how to capture a frame from the Portenta Vision Shield Camera module and save the output as a bitmap image. It will allow you to see the output directly on your computer without using any third party tool.

Goals

Capturing a frame from the camera.

Make the bitmap binary file with the correct settings.

Save the bitmap on an SD Card.

Visualize the captured image on your computer.

Required Hardware and Software

[Portenta H7](#)[Help](#)

USB-C® cable

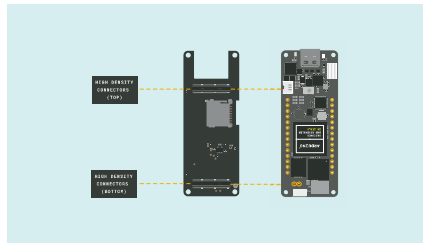
Micro SD card

Arduino IDE 1.8.10+, Arduino IDE 2.0+ or Arduino-cli

Instructions

1. The Setup

Connect the Portenta Vision Shield to your Portenta H7 as shown in the figure. The top and bottom high density connectors are connected to the corresponding ones on the underside of the H7 board. Plug in the H7 to your computer using the USB-C® cable.



The Camera

The Vision Shield Rev.1 uses the **Himax HM-01B0 (320x240) camera module**, the Vision Shield Rev.2 uses the **Himax HM0360 (640x480) camera module**, both output data are grayscale with 8 bits per pixel (bpp). It is important to have this in mind as the `.bmp` (bitmap) format has some needed configuration depending on the data being used.

Inside the sketch, you can use these libraries to access the camera APIs, also compatible with the [Arduino Nicla Vision](#)

```
1 #include "camera.h" // Mu
2
3 //For the Vision Shield R
4 #include "himax.h" // AF
5
6 //For the Vision Shield R
7 #include "hm0360.h" // AF
```



Left uncommented the library
of your Vision Shield version.

Bitmap File Format

The bitmap binary file needs to contain some information in order to tell the computer for example the resolution of the picture and the bit-depth (bpp). Bit depth refers to the color information stored in the image. The higher the bit depth of an image, the more colors it can store. As the bit depth increases, the file size of the image also increases, because more color information has to be stored for each pixel in the image.

The following table shows all the headers, the size of its buffer, offsets, the settings that are being used with their details:

Name	Size	Details
DIB	14 Bytes	Bitmap information, setting the size of the file.
File Header	40 Bytes	This header requires the resolution and the bpp
Palette	1025	This header is

Name	Size	Details
		≤ 8, setting the grayscale
Image data	76800 Bytes	The raw image data, in this case each pixel has 8 bits (1 Byte) and the resolution is 320 x 240, with no compression

The final size of the file is **77.879 KB**.

2. The Sketch

You can find the sketch on the latest version of the

[Arduino_Pro_Tutorials](#) at

```
examples > Vision Shield to SD  
Card bmp >  
visionShieldBitmap.ino
```



You can see the full sketch in the [Full sketch section](#)

First you need to include the needed libraries

```
1 #include "SDMMCBlockDevi  
2 #include "FATFileSystem.  
3  
4 #include "camera.h" // A  
5  
6 /*-----Uncomment the lib  
7  
8 //#include "himax.h" //  
9 // OR  
10 #include "hm0360.h" // A
```

Then define the following objects with their respective constructor (`blockDevice` and `fileSystem` objects). needed for getting access

```

1  #include "SDMMCBlockDevi
2  #include "FATFileSystem.
3  SDMMCBlockDevice blockDe
4  mbed::FATFileSystem file
5
6  #include "camera.h" // A
7
8  /*-----Uncomment the lib
9
10 // #include "himax.h" //
11 // HM01B0 himax;
12
13 // OR
14
15 #include "hm0360.h" // A
16 HM0360 himax;
17
18 Camera cam(himax);
19
20 FrameBuffer frameBuffer;

```

For the bitmap headers binary file you will need some information like the resolution of the image, the bits per pixel and more; so you can define your settings as shown:

```

1  // Settings for our setu
2  #define IMAGE_HEIGHT (un
3  #define IMAGE_WIDTH (uns
4  #define IMAGE_MODE CAMER
5  #define BITS_PER_PIXEL (
6  #define PALETTE_COLORS_A
7  #define PALETTE_SIZE (u
8  #define IMAGE_PATH "/fs/
9
10 // Headers info
11 #define BITMAP_FILE_HEAD
12 #define DIB_HEADER_SIZE
13 #define HEADER_SIZE (BIT

```

The program has the following functions declared:

```
void mountSDCard()
```

```
void
setFileHeaders(bitmapFile
Headers, bitmapDIBHeader,
fileSize)
```

```
void
setColorMap(colorMap)
```

```
saveImage(imageData,
imagePath)
```

To mount the SD Card you will use the following function in the sketch:

```
1 // Mount File system blo
2 void mountSDCard(){
3     int error = fileSyst
4     if (error){
5         Serial.println("
6         int formattingEr
7         if (formattingEr
8         Serial.print
9         while (1);
10    }
11 }
12 }
```

The function to capture the frame from the camera:

```
1 // Get the raw image data
2 unsigned char * captureIm
3     if (cam.grabFrame(fra
4         return frameBuffe
5     } else {
6         Serial.println("c
7         while (1);
8     }
9 }
```

To manipulate the file you will need to set the headers on the binary information as follows:

```
1 // Set the headers data
2 void setFileHeaders(unsig
3 // Set the headers t
4 memset(bitmapFileHea
5 memset(bitmapDIBHead
6
7 // File header
8 bitmapFileHeader[0]
9 bitmapFileHeader[1]
10 bitmapFileHeader[2]
11 bitmapFileHeader[3]
12 bitmapFileHeader[4]
13 bitmapFileHeader[5]
14 bitmapFileHeader[10]
15
16 // Info header
17 bitmapDIBHeader[0] =
18 bitmapDIBHeader[4] =
19 bitmapDIBHeader[5] =
20 bitmapDIBHeader[8] =
21 bitmapDIBHeader[9] =
22 bitmapDIBHeader[14]
23 }
```

Take a look at the section

[Bitmap File Format](#) to better



understand the file headers
that are created with this
function

In this case that our image
(320x240) is 8 bits per pixel and
grayscale on the bitmap rules you
need to define the color table (color
map) to assign an specific RGB color
for our 8 bit color. On the following
function it sets the color map as a
grayscale of RGB colors from
[R:0x00 G:0x00 B:0x00] to [R:0xFF
G:0xFF B:0xFF]

```

1 void setColorMap(unsigned
2 //Init the palette w
3 memset(colorMap, (un
4
5 // Gray scale color
6 for (int i = 0; i <
7     colorMap[i * 4]
8     colorMap[i * 4 +
9     colorMap[i * 4 +
10 }
11 }

```

The function in charge to save the image will use the previous functions to set the headers and write the buffers into the file

`image.bmp`

```

1 // Save the headers and
2 void saveImage(unsigned
3 int fileSize = BITMA
4 FILE *file = fopen(i
5
6 // Bitmap structure
7 unsigned char bitmap
8 unsigned char bitmap
9 unsigned char colorM
10
11 setFileHeaders(bitma
12 setColorMap(colorMap
13
14 // Write the bitmap
15 fwrite(bitmapFileHea
16 fwrite(bitmapDIBHead
17 fwrite(colorMap, 1,
18 fwrite(imageData, 1,
19
20 // Close the file st
21 fclose(file);
22 }

```

Then to have visual feedback lets add a blink function to make 3 blinks after the photo is taken, once the blue LED is ON it means the picture was taken


```
1 void countdownBlink(){
2     for (int i = 0; i < 6
3         digitalWrite(LEDG
4         delay(500);
5     }
6     digitalWrite(LEDG, HI
7     digitalWrite(LEDB, LO
8 }
```

Now that you have all the functions to be used, inside the `setup()` its call them only once after the board restarts.

```
1 void setup(){
2     Serial.begin(115200)
3     while (!Serial && mi
4
5     Serial.println("Moun
6     mountSDCard();
7     Serial.println("SD C
8
9     // Init the cam QVGA
10    if (!cam.begin(CAMER
11        Serial.println("
12    }
13
14    countdownBlink();
15    Serial.println("Fetc
16    unsigned char *image
17
18    Serial.println("Savi
19    saveImage(imageData,
20
21    fileSystem.unmount()
22    Serial.println("Done
23 }
```

The `loop()` is empty, as it only does one shot when the Serial Monitor is open or after 5 seconds passed after boot.

Select the right serial port on your IDE and upload the Arduino sketch to your Portenta H7.

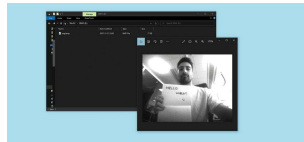
4. Try It Out

Insert a micro SD Card into the Portenta Vision Shield.

Connect the Portenta Vision Shield to the Portenta H7.

Once the sketch is uploaded, open the Serial Monitor or wait 5 seconds: you should see that everything is fine and the capture has been taken.

Once the capture is saved, remove the SD Card and plug it into a computer/phone with an SD Card reader, open the storage unit, look for a bitmap called `image.bmp` and open it to check the result. You will be able to see a grayscale image on your device's image viewer.



Output bitmap example

Full Sketch



```

1  #include "SDMMCBlockDe
2  #include "FATFileSyste
3  SDMMCBlockDevice block
4  mbed::FATFileSystem fi
5
6  #include "camera.h" /,
7
8  /*-----Uncomment the I
9
10 // #include "himax.h"
11 // HM01B0 himax;
12
13 #include "hm0360.h" /,
14 HM0360 himax;
15
16 Camera cam(himax);
17
18 FrameBuffer frameBuffe
19
20 // Settings for our se
21 #define IMAGE_HEIGHT (
22 #define IMAGE_WIDTH (t
23 #define IMAGE_MODE CAM
24 #define BITS_PER_PIXEL
25 #define PALETTE_COLORS
26 #define PALETTE_SIZE
27 #define IMAGE_PATH "/1
28

```

Conclusion

In this tutorial you learned how to capture frames with your Portenta Vision Shield's Camera in the Arduino IDE, encode it with the bitmap standards and save it to an SD Card.

Next Steps

With this sketch format, you could easily add some code in the `loop()` in order to capture 1 new frame each second and save it with a name, for example

```
image<time/count>.bmp
```

**Suggest
changes**

The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page [here](#).

**Need
support?**

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino](#)
[Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?